

APARA Simulator (engine_isp)
Source Fixes Applied During Compiler Verification
What Changed, and Why Each Change Was Necessary

APARA C Compiler Project — Verification & Audit

July 19, 2026

Contents

1	Introduction: Why These Edits Were Made	2
1.1	Context	2
1.2	Why the simulator source had to be edited	2
1.3	Summary of all changes	3
2	The Six Changes in Detail	4
2.1	<code>\$fsqrt</code> executed as an unimplemented stub (NEW)	4
2.2	Float casts fell into an unimplemented error branch (RE-APPLIED)	5
2.3	<code>int↔float</code> cast dispatch called the helpers swapped (RE-APPLIED)	5
2.4	Scalar 32-bit casts misclassified as vector casts (NEW)	6
2.5	Unsigned <code>\$vreduce</code> sign-extended its lanes (NEW)	6
2.6	<code>cast_int_to_float</code> : ternary promotion of negatives (RE-APPLIED)	7
3	Checked and Found Already Correct (No Change Made)	9
4	Process Note: Keeping Fixes in Source	10

Chapter 1

Introduction: Why These Edits Were Made

1.1 Context

The APARA C compiler is now feature-complete (integer C, floating point, function pointers, variadic functions, >4 arguments). To find any remaining defects before hardware bring-up, we ran a **differential verification campaign**: 1,013 C programs (a 13-test directed battery plus 1,000 randomly generated whole programs), each compiled twice — once by the APARA compiler and executed on the `engine_isp` simulator, and once natively by `gcc`. Every `results[]` slot must match `gcc` bit-for-bit. The generated programs are constructed to be free of undefined behaviour, so *every mismatch is a genuine toolchain defect*, either in the compiler or in the simulator.

A coverage auditor additionally proves that the campaign leaves nothing untouched: all 102 (instruction $\times$ sub-operation $\times$ type-tag) combinations the compiler can emit are exercised by at least one passing, `gcc`-verified program.

1.2 Why the simulator source had to be edited

Six defects found by this campaign are in the *simulator*, not the compiler. They could not be worked around from the compiler side:

- Three make **ISA-documented instructions produce silently wrong values** (`$fsqrt` returns 0; unsigned `$vreduce` treats lanes as signed; scalar float $\rightarrow$ int casts lose their sign extension). A compiler cannot emit correct code for an instruction the simulator executes incorrectly — and “silently wrong” is the worst failure mode for hardware validation, because the RTL would be judged against a wrong reference.
- Three are **re-applications of the July 17–18 floating-point fixes** (cast dispatch swap, the unimplemented float-cast branch, and the `cast_int_to_float` promotion bug). Those had been fixed and deployed as binaries during the FP campaign, but the source tree had been left pristine; a later rebuild silently reverted them (Chapter 4). They are now fixed *in source*.

Each edit is **minimal and local**: it uses machinery that already exists in the file (e.g. `$fsqrt` is routed through the existing ALU dispatch, which already implements `fp_sqrt`), changes no interfaces, and alters no behaviour outside the defective case. After *every* rebuild the full regression gate was re-run (feature_sweep 16/16, universal 21/21, pointer_bugs 15/15, fp01-09 50/50), and the final campaign result is **613 PASS / 0 FAIL / 0 HANG** with 102/102 coverage.

1.3 Summary of all changes

#	File / function	Defect	Why the edit was necessary
1	MachineRun.cpp McodeFsqrtInstruction::Execute	<code>\$fsqrt</code> was an unimplemented instruction returning 0	<code>\$fsqrt</code> is in the ISA and the assembler grammar; every <code>sqrtsqrtf</code> silently produced 0
2	McodeExecute.cpp ___execute_cast_operation	float casts fell into an error on branch, output register left <i>uninitialized</i>	all f32/f64 $\leftrightarrow$ int casts landed host-memory garbage in registers (re-applied Jul-17 fix)
3	McodeOperations.cpp ___cast_operation___	int $\leftrightarrow$ float dispatch called the two helpers swapped	every conversion read its operand as the wrong representation (re-applied Jul-17 fix)
4	McodeOperations.cpp ___cast_operation___	scalar 32-bit casts misclassified as <i>vector</i> casts	the result was masked to 32 bits, destroying the sign extension of negative float $\rightarrow$ int results
5	McodeOperations.cpp __vreduce_operation__	unsigned <code>\$vreduce</code> sign-extended its lanes	<code>\$vu8/\$vu16/\$vu32</code> reductions summed/compared lanes as negative numbers
6	McodeFpuUtils.cpp cast_int_to_float	C ternary promotion turned negative ints into $2^{64} - n$	every <i>negative</i> int $\rightarrow$ float cast was wrong (re-applied Jul-18 fix)

Chapter 2

The Six Changes in Detail

2.1 \$fsqrt executed as an unimplemented stub (NEW)

File / function: MachineRun.cpp, McodeFsqrtInstruction::Execute

Symptom. Every \$fsqrt produced 0: (long long)sqrt(4.0) returned 0 instead of 2. Found by directed test d08_fsqrt_float.c; the simulator printed “McodeFsqrtInstruction yet to be implemented” and set the output to 0.

Root cause. The Execute body was a placeholder. All the machinery already existed: the constructor sets opcode __FSQRT, and __alu_operation__ already dispatches __FSQRT to fp_sqrt — the instruction class simply never invoked it.

Why we edited it. \$fsqrt is a documented ISA instruction that the compiler lowers sqrt()/sqrtf() to; without this fix the instruction cannot be validated at all, and a hardware implementation would have no correct simulator reference. The edit only *connects* the stub to the existing, already-tested ALU execution path — no new arithmetic code was written.

Before:

```
1 void McodeFsqrtInstruction::Execute      (McodeMachine* mc)
2 {
3     McodeRoot::Error("McodeFsqrtInstruction yet to be implemented", this);
4     this->McodeInstruction::Set_Out_Arg (0, 0);
5     return;
6 }
```

After:

```
1 void McodeFsqrtInstruction::Execute      (McodeMachine* mc)
2 {
3     uint64_t ovalue;
4     ___execute_alu_operation___ (this->McodeInstruction::Get_Opcode(),
5                                 this->Get_Dest_Type(),
6                                 this->Get_Src_Type(),
7                                 this->McodeInstruction::Get_In_Arg(0),
8                                 0, 0, 0,
9                                 ovalue);
10    this->McodeInstruction::Set_Out_Arg (0, ovalue);
11    if(__global_verbose_flag)
12        McodeRoot::Info(Int64ToString (this->McodeInstruction::Get_Address()) + ":
13    Fsqrt!");
14    return;
15 }
```

Verification. sqrt(4.0), sqrt(2.25), sqrtf(9.0f) etc. all bit-match gcc (d08 = 8/8 golden checks); run-log shows 0x4010000000000000 → 0x4000000000000000 (4.0 → 2.0, exact).

2.2 Float casts fell into an unimplemented error branch (RE-APPLIED)

File / function: McodeExecute.cpp, `___execute_cast_operation___`

Symptom. Any `$cast` with a float type on either side printed “Float conversions in cast not supported as yet.” and left the output array *uninitialized* — host-process stack garbage (values such as `0x7ffd83e474a0`) landed in the destination register.

Root cause. The complete implementation (which forwards to `___cast_operation___`, and that function dispatches `int↔float` itself) was guarded by `if (neither type is float);` float casts took the error branch.

Why we edited it. Floating-point support in the compiler is verified bit-exactly against gcc (fp01–fp09), and every `float↔int` conversion goes through `$cast`. Reading uninitialized memory is also nondeterministic — results changed from run to run, which would make any hardware-vs-simulator comparison meaningless. The guard was removed so every cast takes the one real implementation; no conversion logic was added here.

Before:

```

1  if(!dest_type.Get_Float_Flag()
2      && !src_type.Get_Float_Flag())
3  {
4      /* ... full implementation ... */
5  }
6  else
7  {
8      McodeRoot::Error ("Float conversions in cast not supported as yet.", NULL);
9  }
```

After:

```

1  if(1) /* every cast takes the main path; ___cast_operation___
2      dispatches int↔float itself */
3  {
4      /* ... full implementation (unchanged) ... */
5  }
```

Verification. fp01–fp09 (50/50 golden checks, bit-exact incl. rounding); campaign-wide float casts in 600+ random programs.

2.3 int↔float cast dispatch called the helpers swapped (RE-APPLIED)

File / function: McodeOperations.cpp, `___cast_operation___`

Symptom. Every `int→float` and `float→int` conversion produced garbage: each helper interpreted its operand as the wrong representation (integer bits read as IEEE and vice versa).

Root cause. The dispatch was inverted: an *integer source* was sent to `cast_float_to_int`, and a float source with integer destination to `cast_int_to_float`.

Why we edited it. This is a plain transposition bug — the two correctly implemented helpers were wired to the opposite conditions. The edit swaps the two call sites back; nothing else changes.

Before:

```

1  if(!src_type.Get_Float_Flag())
2      w = cast_float_to_int (src_type, dest_type, v);
3  else if(!dest_type.Get_Float_Flag())
4      w = cast_int_to_float (src_type, dest_type, v);
```

```

5         else
6             w = cast_float_to_float (src_type, dest_type, v);

```

After:

```

1         if(!src_type.Get_Float_Flag())
2             w = cast_int_to_float (src_type, dest_type, v);
3         else if(!dest_type.Get_Float_Flag())
4             w = cast_float_to_int (src_type, dest_type, v);
5         else
6             w = cast_float_to_float (src_type, dest_type, v);

```

Verification. Same as change 2 (all casts flow through here).

2.4 Scalar 32-bit casts misclassified as vector casts (NEW)

File / function: McodeOperations.cpp, __cast_operation__

Symptom. (int)(-6.0f) stored 0x00000000FFFFFFFA instead of the sign-extended 0xFFFFFFFFFFFFFFFA; regression test fp03 caught it. The conversion itself was correct (`cast_float_to_int` returns the full sign-extended 64-bit value) — the result was then masked back to 32 bits.

Root cause. Vector-ness was inferred from the number of unpacked input elements, `in_vals.size() > 1`. But `Break_Vector` splits even a *scalar* 32-bit source register into two 32-bit entries, so every scalar cast with a 32-bit source was treated as a vector cast, and vector casts mask each output to the destination lane width.

Why we edited it. The type words themselves carry an explicit `Vector_Flag`; inferring vector-ness from a size heuristic contradicts the instruction’s own declared type and cannot distinguish “scalar f32” from “vector of two f32 lanes”. Using the declared flags makes the scalar path take the unmasked (sign-preserving) result, and leaves true vector casts exactly as before.

Before:

```

1     int is_vector_cast = (in_vals.size() > 1);

```

After:

```

1     int is_vector_cast = (src_type.Get_Vector_Flag() && dest_type.Get_Vector_Flag());

```

Verification. fp03 (negative float→int) restored to 4/4; full gate 61/61; d02 cast matrix 8/8.

2.5 Unsigned \$vreduce sign-extended its lanes (NEW)

File / function: McodeOperations.cpp, __vreduce_operation__

Symptom. `__vreduce_vu8(0x01f2030405060708)` returned 0x14 (20) instead of 0x114 (276): the 0xf2 lane was summed as -14 instead of 242. `__vreduce_max_vu8` likewise picked the wrong maximum. Found by directed test `d06.vreduce.c`.

Root cause. A shared lane variable `r` was computed with `Sign_Extend_64` *before* the signed/unsigned branch; the signed branch shadows it with its own (correct) sign-extended copy, but the *unsigned* branch used the outer, sign-extended value.

Why we edited it. The entire point of the `$vu*` type tags is zero-extension of lanes; with this bug the unsigned reductions were indistinguishable from the signed ones, i.e. half of the `$vreduce` ISA surface was unimplemented in effect. The lane value arriving from `Break_Vector` is already masked to the lane width, so the unsigned branch simply uses it directly. The signed branch is untouched.

Before:

```

1      // sign extend to 64-bits.
2      int64_t r = (int64_t) Sign_Extend_64(src_type.Get_Nbits()-1, ele);
3      if(signed_flag)
4      {
5          int64_t r = (int64_t) Sign_Extend_64(...); // shadows outer r
6          ...
7      }
8      else
9      {
10         ... uses the OUTER, sign-extended r ...
11     }

```

After:

```

1      uint64_t r = ele; // zero-extended: ele is already lane-masked
2      if(signed_flag)
3      {
4          int64_t r = (int64_t) Sign_Extend_64(...); // unchanged
5          ...
6      }
7      else
8      {
9          ... unsigned branch now sums/compares zero-extended lanes ...
10     }

```

Verification. d06 = 8/8 golden checks covering +/\$max at vi8/vu8/vi16/vu16/vi32/vu32 vs gcc.

2.6 cast_int_to_float: ternary promotion of negatives (RE-APPLIED)

File / function: McodeFpuUtils.cpp, cast_int_to_float

Symptom. (float)(-3) became $2^{64} - 3 \approx 1.8 \times 10^{19}$ as a double, then saturated on any conversion back. All *positive* int→float casts were unaffected, which is how the bug originally escaped notice.

Root cause. In cond ? (u & 0xffffffff) : (int) u the C conditional operator promotes both arms to a common type — here the unsigned arm's uint64_t — so the signed arm's negative int value was converted back to a huge unsigned number before becoming a double.

Why we edited it. This is a classic C promotion pitfall, invisible in code review but fatal for every negative operand. The replacement makes the signedness handling explicit (branch, then sign-extend from the actual source width) — it also fixes the old unsigned arm's hard-coded 32-bit mask, which mishandled unsigned 8/16/64-bit sources.

Before:

```

1      double x = (src_type.Get_Unsigned_Flag() ? (u & 0xffffffff) : (int) u);

```

After:

```

1      double x;
2      if(src_type.Get_Unsigned_Flag())
3      {
4          x = (double) (u & __mmask__(snbits));
5      }
6      else
7      {
8          int64_t sv = ((int64_t) (u << (64 - snbits))) >> (64 - snbits);
9          x = (double) sv;
10     }

```


Verification. fp03 (positive and negative int→float, $\times$, $\div$, $+$) = 4/4 vs gcc; hand-written `$cast` micro-test from the original Jul-18 session.

Chapter 3

Checked and Found Already Correct (No Change Made)

For completeness, the campaign also re-audited areas with a history of defects; the following are **correct in the current source** and were left untouched:

- `McodeFpu.hpp __float32_sub__ / __float64_sub__`: the July 17 `fp_sub` defects (bit-pattern subtraction at 32-bit; ‘+’ instead of ‘−’ at 64-bit) are *not present* — both macros convert through double and subtract correctly.
- The unsigned scalar ALU path (`__uexec_64__` on raw `uint64_t` + `CastToU64`) is correct, including unsigned divide and logical right shift. The older build’s `$u64`-zeroing defect no longer exists; consequently the compiler now emits `($u64)` for unsigned `/`, `%` (its divide step) and `>>`, all verified against gcc (including a logical shift by 63).
- The July 5 backward-`$call` sign-extension fix is present; every call to an earlier-defined function in the 61-test gate is a backward call.

Two *behavioural* findings in the assembler worth knowing (not changed):

- `$pack` requires `packed_nbits % word_nbits == 0` (`McodeClasses.hpp:412`); violations are reported only as the generic “there were invalid instructions” with no per-instruction message.
- `mcode_align` reports lane-placement failures (`>4` load/store or `>1` divide/sqrt per bundle) on stderr but *still emits the bundle* with the control-transfer instruction not moved to lane 0 — downstream this caused a return into the middle of a bundle. The compiler’s bundler now enforces these limits itself, and our scripts no longer discard align/assemble stderr.

Chapter 4

Process Note: Keeping Fixes in Source

Changes 2, 3 and 6 were originally made during the July 17–18 floating-point campaign, but only the *binaries* were deployed; the source tree was left pristine. When the toolchain was rebuilt on July 19 (for the `$fsqrt` fix), the rebuild regenerated all binaries from the unfixed source and silently reverted those three fixes. The regression gate caught this immediately (fp03 failing, 60/61), and all fixes were re-applied *in source* with explanatory comments.

Two safeguards are now in place:

- `testing/fuzz1000/check_engine_fixes.sh` greps the simulator source for all six fixes; it is run before and after any rebuild.
- The full regression gate (61 golden tests) plus, when relevant, the fuzz campaigns are re-run after *every* toolchain rebuild.

Recommendation for the upstream tree: absorbing the six diffs in Chapter 2 brings the upstream simulator in line with the reference against which the compiler is verified. Each is minimal, local, and documented at the change site with a comment beginning “Re-applied fix” or referring to the fuzz1000 campaign.